GeoXGB: Mathematical Architecture of Geometry-Aware Gradient Boosting

1 Setup and Notation

Let $X \in \mathbb{R}^{n \times d}$ be the training feature matrix and $y \in \mathbb{R}^n$ the target vector. Throughout, subscript i indexes samples and superscript (j) indexes features. We write $x_i \in \mathbb{R}^d$ for the i-th row of X.

1.1 Whitening

HVRT operates on *whitened* (z-scored) features. The whitener computes per-column statistics:

$$z_i^{(j)} = \frac{x_i^{(j)} - \mu^{(j)}}{\sigma^{(j)} + \epsilon}, \qquad \mu^{(j)} = \frac{1}{n}\sum_{i=1}^{n} x_i^{(j)}, \quad \sigma^{(j)} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\big(x_i^{(j)} - \mu^{(j)}\big)^2}, \tag{1}$$

with $\epsilon = 10^{-8}$. This gives the whitened matrix $Z \in \mathbb{R}^{n \times d}$ with each column approximately $\mathcal{N}(0, 1)$. The whitener parameters (μ, σ) are computed *once* at the initial HVRT fit and frozen for all subsequent refits.

1.2 Inverse Transform

Synthetic samples generated in z-space are mapped back to the original feature space via

$$\hat{x}^{(j)} = z^{(j)} \cdot \sigma^{(j)} + \mu^{(j)}. \tag{2}$$

2 The Geometry Target

2.1 Pairwise Cooperation Target (HVRT)

For the default HVRT partitioner ($d \leq 50$), the geometry target $T \in \mathbb{R}^n$ is the *pairwise cooperation statistic*:

$$T_i = \mathrm{zscore}\left(\sum_{a<b} \mathrm{zscore}\big(z_i^{(a)} \cdot z_i^{(b)}\big)\right), \tag{3}$$

where $\mathrm{zscore}(v) = (v - \bar{v})/\sigma_v$ standardises a vector to zero mean and unit variance (population).

Remark 1 (Algebraic identity)**.** T is proportional to the second elementary symmetric polynomial:

$$\sum_{a<b} z_i^{(a)} z_i^{(b)} = \frac{1}{2}\Big[\Big(\sum_j z_i^{(j)}\Big)^2 - \sum_j \big(z_i^{(j)}\big)^2\Big] = \frac{S_i^2 - Q_i}{2} = e_2(\boldsymbol{z}_i),$$

where $S_i = \sum_j z_i^{(j)}$ and $Q_i = \|\boldsymbol{z}_i\|_2^2$. The z-score normalisation within each pair column absorbs the scale but preserves the rank ordering, so T partitions samples along iso-e_2 hyperboloids in z-space. Computational cost: $\mathcal{O}(n \cdot d^2)$.

Remark 2 (Noise invariance)**.** If each feature is corrupted by independent additive noise $\tilde{z}^{(j)} = z^{(j)} + \varepsilon^{(j)}$ with $\mathbb{E}[\varepsilon^{(j)}] = 0$, then

$$\mathbb{E}[\tilde{S}^2 - \tilde{Q}] = \mathbb{E}[S^2 - Q] + d\sigma_\varepsilon^2 - d\sigma_\varepsilon^2 = \mathbb{E}[S^2 - Q].$$

The additive noise bias cancels exactly because both S^2 and Q are degree-2 in the features. This is the HVRT noise invariance theorem.

2.2 Alternative Targets

- **HART**: $T_i = \text{zscore}\big(\frac{1}{2}(\|z_i\|_1^2 - \|z_i\|_2^2)\big) = \text{zscore}\big(\sum_{a<b} |z_i^{(a)}||z_i^{(b)}|\big)$. Degree-2, noise-invariant, $\mathcal{O}(n \cdot d)$.
- **PyramidHART**: $T_i = \text{zscore}(|S_i| - \|z_i\|_1)$. Degree-1, outlier-cancelling, $\mathcal{O}(n \cdot d)$.
- **FastHART** ($d > 50$): $T_i = \text{zscore}(S_i)$. Degree-1, $\mathcal{O}(n \cdot d)$.

3 Blended Target and Partition Tree

3.1 Y-Weight Blending

The pure geometry target T is blended with the y-signal (raw targets on initial fit, or residual gradients on refit) to bias the partition tree towards regions that are both geometrically interesting *and* predictively important.

$$y_{\text{norm}} = \frac{y - y_{\min}}{y_{\max} - y_{\min}} \in [0,1]^n, \tag{4}$$

$$y_{\text{dev}} = \big|y_{\text{norm}} - \text{med}(y_{\text{norm}})\big|, \tag{5}$$

$$y_{\text{comp}} = \text{zscore}(y_{\text{dev}}), \tag{6}$$

$$t_{\text{blend}} = \text{zscore}\big((1 - w_y)\ \text{zscore}(T) + w_y\, y_{\text{comp}}\big), \tag{7}$$

where $w_y \in [0,1]$ is the `y_weight` parameter.

Remark 3. y_{comp} measures distance from the median response: extreme y values (both high and low) get large y_{comp}. This is deliberate — the partition tree should separate extreme-response regions regardless of sign, because both tails carry the most predictive information for gradient boosting.

3.2 Partition Tree

The blended target t_{blend} is used as the splitting criterion for a variance-reduction decision tree on Z. The tree greedily minimises

$$\underset{j,\theta}{\text{argmin}} \sum_{i \in L} (t_i - \bar{t}_L)^2 + \sum_{i \in R} (t_i - \bar{t}_R)^2, \qquad L = \{i : z_i^{(j)} \le \theta\},\ \ R = \{i : z_i^{(j)} > \theta\}, \tag{8}$$

producing a set of P leaf partitions $\{\mathcal{P}_1, \ldots, \mathcal{P}_P\}$. Each sample i is assigned to exactly one partition $p(i)$.

The partition tree operates on binned features (16 bins) for speed, using histogram-based split enumeration. The tree depth and leaf sizes are auto-tuned: `min_samples_leaf` $= 10$, `max_partitions` $= \min(1500,\ 3n/20)$.

4 Reduce–Expand Resampling

Given the partition structure $\{(\mathcal{P}_p, Z_p)\}_{p=1}^P$, HVRT performs a *reduce* step (discard uninformative samples) followed by an optional *expand* step (generate synthetic samples).

4.1 Budget Allocation

Each partition p receives a budget b_p of how many samples to retain (or generate). Two modes:

- **Proportional** (default): $b_p \propto |\mathcal{P}_p|$.
- **Variance-weighted**: $b_p \propto \bar{v}_p$ where $\bar{v}_p = \frac{1}{|\mathcal{P}_p|} \sum_{i \in \mathcal{P}_p} \frac{1}{d} \sum_j |z_i^{(j)}|$ is the mean absolute deviation within partition p.

The total budget is $n_{\text{keep}} = \lfloor n \cdot r_{\text{eff}} \rfloor$, where r_{eff} is the effective reduce ratio (see §**??**).

4.2 Variance-Ordered Reduction

Within each partition p of size n_p with budget $b_p < n_p$, the **variance-ordered** selector retains samples that maximise geometric diversity:

1. Compute the $n_p \times n_p$ pairwise squared-distance matrix $D_{ij} = \|z_i - z_j\|_2^2$ for $i, j \in \mathcal{P}_p$.
2. For each i, find the k-nearest neighbours ($k = 5$) and compute $v_i = \text{Var}(\{D_{ij}\}_{j \in \text{kNN}(i)})$ — the local distance variance.
3. Sort by v_i descending and retain the top b_p samples.

High local-distance-variance points lie near partition boundaries or in heterogeneous regions of z-space. Retaining them preserves the geometric "skeleton" of each partition while discarding interior samples that are well-represented by their neighbours.

Remark 4 (Scalability)**.** When $n_p > 400$, a random subsample of 400 candidates is drawn first, variance-ordering is applied to the subsample, and the selected indices are mapped back. When $b_p/n_p \geq 0.65$, the selection degenerates to random sampling (Fisher–Yates) since the quality gain over random is negligible at high retention fractions.

4.3 Expansion: Synthetic Sample Generation

When `expand_ratio` > 0 or $n <$ `min_train_samples`, HVRT generates $n_{\text{syn}} = \lfloor n \cdot e \cdot \hat{\nu} \rfloor$ synthetic samples in z-space, where e is the expand ratio and $\hat{\nu}$ is the noise modulation (§**??**).

Synthetic samples are generated per-partition via kernel density estimation. For the **Epanechnikov** kernel (default): within partition p with samples $\{z_i\}_{i \in \mathcal{P}_p}$, draw a centre z_c uniformly at random from the partition's training points, then sample

$$z_{\text{syn}} = z_c + h_p \cdot u, \qquad u \sim \text{Epanechnikov}(d), \tag{9}$$

where h_p is the per-partition bandwidth (Scott's rule: $h_p = n_p^{-1/(d+4)}\, \hat{\sigma}_p$).

The synthetic z-vectors are inverse-transformed back to feature space, giving $X_{\text{syn}} \in \mathbb{R}^{n_{\text{syn}} \times d}$.

4.4 Y-Assignment for Synthetic Samples

Synthetic samples have features but no target values. These are assigned via **inverse-distance-weighted** k**-NN** ($k = 3$) in z-space:

$$\hat{y}_{\text{syn}} = \frac{\sum_{j \in \text{kNN}} w_j\, y_j}{\sum_{j \in \text{kNN}} w_j}, \qquad w_j = \frac{1}{\|z_{\text{syn}} - z_j\|_2 + \epsilon}, \tag{10}$$

where the nearest-neighbour search is over the *reduced* real training set $\{(z_i, y_i)\}_{i \in \mathcal{R}}$ and $\epsilon = 10^{-8}$.

The final training matrix for each boosting interval is:

$$\tilde{X} = \begin{bmatrix} X_{\text{red}} \\ X_{\text{syn}} \end{bmatrix} \in \mathbb{R}^{(n_{\text{red}} + n_{\text{syn}}) \times d}, \qquad \tilde{y} = \begin{bmatrix} y_{\text{red}} \\ \hat{y}_{\text{syn}} \end{bmatrix}. \tag{11}$$

5 The Boosting Loop

5.1 Initialisation

1. Compute initial prediction:
   - **Regression**: $\hat{y}_0 = \bar{y}$ (mean of training targets).
   - **Classification**: $\hat{y}_0 = \log \frac{\bar{y}}{1-\bar{y}}$ (log-odds).
2. Set all predictions: $\hat{y}_i^{(0)} = \hat{y}_0$ for all i.
3. Run initial HVRT fit on (X, y) using raw targets as the y-signal.
4. Reduce and optionally expand, producing $(\tilde{X}^{(0)}, \tilde{y}^{(0)})$.
5. Set resampled predictions: $\tilde{p}_i^{(0)} = \hat{y}_0$ for all $i \in \tilde{X}^{(0)}$.

5.2 Round-by-Round Gradient Descent

For round $t = 1, \ldots, T$ (`n_rounds`):

Step 1: Compute gradients on the resampled set.

$$g_i^{(t)} = \tilde{y}_i^{(t-1)} - \tilde{p}_i^{(t-1)} \qquad \text{(regression, squared error)} \tag{12}$$

$$g_i^{(t)} = \tilde{y}_i^{(t-1)} - \sigma(\tilde{p}_i^{(t-1)}) \qquad \text{(classification, log-loss)} \tag{13}$$

where $\sigma(v) = 1/(1 + e^{-v})$ is the sigmoid function.

Step 2: Fit a weak learner tree on the resampled data. A decision tree h_t is built on $(\tilde{X}^{(t-1)}, g^{(t)})$ using:

- Pre-computed bin edges from the *full* dataset (64 bins).
- Random split strategy (ExtraTrees-style): for each candidate feature, the split threshold is drawn uniformly from the bin edges.
- Variance-reduction splitting on the gradient values.
- `max_depth`, `min_samples_leaf` control tree complexity.

Step 3: Update predictions.

$$\tilde{p}_i^{(t)} = \tilde{p}_i^{(t-1)} + \eta \cdot h_t(\tilde{x}_i) \qquad \text{(resampled set)} \tag{14}$$

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta \cdot h_t(x_i) \qquad \text{(full training set)} \tag{15}$$

where η is the `learning_rate`.

Remark 5 (Dual prediction tracking)**.** GeoXGB maintains *two* prediction vectors: $\tilde{p}$ on the resampled set (used for gradient computation on the reduced+synthetic data) and $\hat{y}$ on the full training set (used for gradient computation at refit boundaries and for the final model output). Each tree is evaluated on both sets every round: $h_t(\tilde{X})$ and $h_t(X)$. This is $\mathcal{O}(n)$ per round, not $\mathcal{O}(n_{\text{red}})$.

6 Refits: Geometry Re-Survey

Every `refit_interval` rounds (e.g. every 50 rounds), the partition geometry is re-surveyed using the *current gradient landscape*. This is the critical mechanism that couples the static geometry to the evolving loss surface.

6.1 Refit Trigger

At round t where $t \bmod R = 0$ (with $R =$ `refit_interval`):

1. Compute full-data gradients: $g_i^{(\text{full})} = y_i - \hat{y}_i^{(t)}$ (regression) or $g_i^{(\text{full})} = y_i - \sigma(\hat{y}_i^{(t)})$ (classification).
2. Pass $g^{(\text{full})}$ as the new y-signal to HVRT refit.

6.2 What Changes at Refit

The HVRT refit is a *fast path* that reuses:

- The whitened matrix Z (computed once at initial fit).
- The binned feature matrix (computed once at initial fit).
- The geometry target T (computed once at initial fit).

What changes:

1. The y-signal in the blend is now the gradient vector $g^{(\text{full})}$ instead of the raw targets y.
2. The blended target becomes:

$$t_{\text{blend}}^{(t)} = \text{zscore}\big((1 - w_y^{(t)})\, T + w_y^{(t)}\, y_{\text{comp}}(g^{(\text{full})})\big), \tag{16}$$

   where $y_{\text{comp}}(g) = \text{zscore}(|g_{\text{norm}} - \text{med}(g_{\text{norm}})|)$ measures gradient extremality.
3. The partition tree is rebuilt on Z with the new blended target.
4. The expander (KDE parameters) is re-fitted if partitions changed.
5. Reduction and expansion produce a new $(\tilde{X}^{(t)}, \tilde{y}^{(t)})$.

6.3 Geometric Interpretation of Refit

Definition 1 (Refit as gradient-geometry alignment)**.** At round 0, the partition tree separates z-space along iso-T level sets blended with raw-target extremality. The tree knows which samples have high pairwise feature cooperation but knows nothing about which regions the boosted ensemble struggles with.

At refit round t, the gradient $g^{(\text{full})}$ encodes exactly where the ensemble is wrong: large $|g_i|$ means sample i is poorly predicted. The blended target $t_{\text{blend}}^{(t)}$ now separates z-space into regions that are both geometrically coherent *and* where the ensemble has high residual error.

This means:

- Partitions *drift* toward hard-to-predict regions of z-space.
- The reduced set preferentially retains samples with high residuals.
- Synthetic samples are generated near geometrically interesting points that the ensemble cannot yet fit.
- As the ensemble improves, gradients shrink, y_{comp} becomes more uniform, and the partition tree reverts toward pure geometry.

6.4 Adaptive Y-Weight

When `adaptive_y_weight` is enabled, the effective y-weight is scaled by the alignment between the geometry target and the current residuals:

$$w_y^{(t)} = w_y \cdot |\rho(T, g^{(\text{full})})|, \tag{17}$$

where ρ is the Pearson correlation. When the gradients are orthogonal to the geometry ($\rho \approx 0$), $w_y^{(t)} \to 0$ and the partition tree becomes pure geometry. When the gradients are strongly aligned with the geometry ($|\rho| \approx 1$), the full y-weight is applied.

Remark 6. This prevents the gradient signal from corrupting the geometry when the residuals are pure noise (late in training when the ensemble has converged). It is the adaptive mechanism that allows GeoXGB to self-regulate the geometry–gradient coupling.

7 Noise Modulation

7.1 Signal-to-Noise Estimator

At each resample event, GeoXGB estimates the local signal-to-noise ratio $\hat{\nu} \in [0, 1]$:

1. Subsample $m = \min(500, n)$ probe points.
2. For each probe i, find its k-nearest neighbours in z-space ($k = \min(10, n/30)$) and compute the local mean $\bar{y}_i^{(\text{local})} = \frac{1}{k} \sum_{j \in \text{kNN}(i)} y_j$.
3. Compute local-mean variance $\sigma^2_{\text{local}} = \text{Var}(\bar{y}^{(\text{local})})$ and total y-variance $\sigma^2_y = \text{Var}(y)$.
4. SNR ratio: $\text{snr} = \sigma^2_{\text{local}} / \sigma^2_y$.
5. $\hat{\nu} = \text{clamp}\big(\frac{\text{snr} - 0.15}{0.45},\ 0,\ 1\big)$.

Remark 7. $\hat{\nu} = 1$ means pure signal (local averages explain all y-variance). $\hat{\nu} = 0$ means pure noise (local averages are constant — y has no spatial structure in z-space).

7.2 How Noise Modulation Affects the Loop

- **Reduce ratio**: $r_{\text{eff}} = r + (1 - \hat{\nu})(1 - r)$. Noisier data → keep more samples (less aggressive reduction).
- **Expand count**: $n_{\text{syn}} = \lfloor n \cdot e \cdot \hat{\nu} \rfloor$. Noisier data → generate fewer synthetic samples.
- **Noise guard**: when $\hat{\nu} < 0.05$ and $n <$ `min_train_samples`, skip the refit entirely (the gradient is too noisy to guide the geometry).

8 Block Cycling (Large-n Scalability)

When $n > n_{\text{block}}$ (auto-determined or user-specified), the full training set is divided into non-overlapping blocks of size n_{block}.

- A deterministic LCG-based permutation shuffles sample indices.
- At each refit boundary, the boosting loop advances to the next block.
- When all blocks are exhausted, a new epoch begins with a fresh permutation.
- Each block gets a *fresh* HVRT fit (whitening, geometry target, partition tree) since different blocks cover different geometric regions.

$$n_{\text{block}} = \max\big(2000,\ \lfloor \sqrt{n} \cdot 15 \cdot \text{ri_scale} \rfloor\big), \qquad \text{ri_scale} = \text{clamp}\Big(\frac{R}{50},\ 0.5,\ 2.0\Big). \tag{18}$$

Remark 8. Block cycling makes all per-round costs $\mathcal{O}(n_{\text{block}})$ instead of $\mathcal{O}(n)$. The $\sqrt{n}$ scaling ensures blocks grow sub-linearly: at $n = 100{,}000$ with $R = 50$, $n_{\text{block}} \approx 4{,}743$.

9 Complete Algorithm

Algorithm 1 GeoXGB Fit (Regression, Squared Error)

Require: $X \in \mathbb{R}^{n \times d}$, $y \in \mathbb{R}^n$, hyperparameters $\eta, R, r, e, w_y, T_{\max}$

$\hat{y}_0 \leftarrow \bar{y}$; $\quad \hat{\boldsymbol{y}} \leftarrow \hat{y}_0 \cdot \mathbf{1}_n$
Compute whitener: $Z \leftarrow \text{whiten}(X)$
Compute geometry target: $T \leftarrow \text{pairwise_target}(Z)$
$t_{\text{blend}} \leftarrow \text{zscore}\big((1 - w_y)T + w_y\, y_{\text{comp}}(y)\big)$
Build partition tree on $(Z, t_{\text{blend}}) \rightarrow$ partitions $\{\mathcal{P}_p\}$
Fit KDE expander per partition
Reduce: $\mathcal{R} \leftarrow \text{variance_ordered}(\{\mathcal{P}_p\}, \lfloor nr \rfloor)$
Expand: $\mathcal{S} \leftarrow \text{KDE_sample}(\lfloor ne \rfloor)$; $\quad \hat{y}_{\text{syn}} \leftarrow \text{kNN}_{k=3}(Z_{\mathcal{S}}, Z_{\mathcal{R}}, y_{\mathcal{R}})$
$\tilde{X} \leftarrow [X_{\mathcal{R}}; X_{\mathcal{S}}]$; $\quad \tilde{y} \leftarrow [y_{\mathcal{R}}; \hat{y}_{\text{syn}}]$
$\tilde{\boldsymbol{p}} \leftarrow \hat{y}_0 \cdot \mathbf{1}_{|\tilde{X}|}$
for $t = 1$ **to** $T_{\max}$ **do**
  if $t > 0$ and $t \bmod R = 0$ **then**
    $g^{(\text{full})} \leftarrow y - \hat{\boldsymbol{y}}$ {Full-data gradients}
    $w_y^{(t)} \leftarrow w_y \cdot |\rho(T, g^{(\text{full})})|$ {Adaptive y-weight}
    $t_{\text{blend}}^{(t)} \leftarrow \text{zscore}\big((1 - w_y^{(t)})T + w_y^{(t)}\, y_{\text{comp}}(g^{(\text{full})})\big)$
    Rebuild partition tree on $(Z, t_{\text{blend}}^{(t)})$ {Refit: same Z, T; new blend}
    Re-fit KDE if partitions changed
    Re-reduce, re-expand $\rightarrow$ new $(\tilde{X}, \tilde{y})$
    Sync $\tilde{\boldsymbol{p}}$ from $\hat{\boldsymbol{y}}$ for real samples, predict$(\tilde{X}_{\text{syn}})$ for synthetic
    $\tilde{y}_{\text{real}} \leftarrow \tilde{\boldsymbol{p}}_{\text{real}} + g_{\mathcal{R}}^{(\text{full})}$ {Reconstruct targets}
  end if
  $g \leftarrow \tilde{y} - \tilde{\boldsymbol{p}}$ {Gradients on resampled set}
  Fit tree h_t on $(\tilde{X}, g)$ with random splits, depth=D
  $\tilde{\boldsymbol{p}} \leftarrow \tilde{\boldsymbol{p}} + \eta \cdot h_t(\tilde{X})$
  $\hat{\boldsymbol{y}} \leftarrow \hat{\boldsymbol{y}} + \eta \cdot h_t(X)$
end for
$F(x) = \hat{y}_0 + \eta \sum_{t=1}^{T_{\max}} h_t(x)$ {Ensemble prediction}

10 Geometric Evolution Across n

10.1 Small n ($n \lesssim 500$)

- Few partitions (10–50). Each partition covers a broad region of z-space.
- Variance-ordered reduction retains most samples (at $r = 0.7$, keeping 350 of 500).
- Synthetic expansion fills gaps: with $e = 0.1$, generates ∼50 synthetic points.
- The kNN y-assignment uses the full reduced set as reference — accurate because partition interiors are well-sampled.
- Refits shift partitions significantly: $g^{(\text{full})}$ on 500 points is noisy, so the blended target changes substantially each refit.
- Net effect: the geometry acts as a *regulariser* — the partition structure prevents the boosted trees from overfitting to individual noisy samples.

10.2 Moderate n ($n \sim 1{,}000$–$10{,}000$)

- More partitions (50–500). Finer geometry captures local feature interactions.

- Reduction is meaningful: at $r = 0.7$, discarding 30% of 5,000 = 1,500 uninformative interior points. The reduced set of 3,500 preserves the geometric skeleton.
- Synthetic samples concentrate where the KDE has high density — near partition centroids. These provide "consensus" targets that smooth the gradient signal.
- Refits are stable: $g^{(\text{full})}$ on 5,000+ points has low variance, so partition drift is gradual and predictable.
- This is the regime where GeoXGB outperforms XGBoost most consistently: the geometry captures feature-interaction structure that pure tree splitting must discover through many sequential splits.

10.3 Large n ($n \gtrsim 20{,}000$)

- Block cycling activates: each refit sees a different n_{block}-sized slice of the data.
- Each block gets a *fresh* whitener and geometry target. The partitions are block-local, not global.
- The ensemble sees different geometric views at each refit, providing implicit regularisation through geometric diversity.
- At $n = 100{,}000$, $n_{\text{block}} \approx 4{,}700$. Over $T = 1{,}000$ rounds with $R = 50$, the ensemble trains on $\sim$20 blocks $\times$ 3,300 reduced samples = 66,000 unique samples — progressive coverage of the full dataset.
- Prediction cost is $\mathcal{O}(n_{\text{block}})$ per round instead of $\mathcal{O}(n)$, making the total fit cost $\mathcal{O}(T \cdot n_{\text{block}} \cdot d)$.

10.4 Summary: What Each Component Does

Component	**Mathematical Role**	**Effect on Learning**
Whitening	$z = (x - \mu)/\sigma$	Isotropic feature space
Geometry target T	$e_2(z)$ level sets	Feature-interaction structure
Y-weight blend	$(1 - w)T + w\,\lvert y - \text{med}(y)\rvert$	Gradient–geometry coupling
Partition tree	Variance-reduction splits on t_{blend}	Spatial decomposition
Reduction	Retain high-variance boundary points	Remove redundant interior
Expansion	Per-partition KDE sampling	Smooth gradient signal
kNN y-assign	IDW interpolation in z-space	Consistent synthetic targets
Refit	New blend with current gradients	Track evolving loss surface
Adaptive w_y	$w_y \cdot \lvert\rho(T, g)\rvert$	Self-regulate coupling
Noise modulation	Local SNR estimate	Gate expansion/reduction
Block cycling	Rotate n_{block}-sized slices	Scalable geometric diversity